



Önálló laboratórium beszámoló

Távközlési és Mesterséges Intelligencia Tanszék

Készítette: Horváth Benedek
benedekhorvath@edu.bme.hu

Neptun-kód: D86EP7

Ágazat: mérnökinformatikus szak

Konzulens: Németh Felicián
nemethf@tmit.bme.hu

Konzulens: Lévai Tamás
levait@tmit.bme.hu

Téma címe: Médiaátvitel QUIC felett

Feladat:

A féléves munka célja a Media over QUIC (MoQ) protokollhoz tartozó moq-rs implementáció megfigyelhetőségi (observability) lehetőségeinek feltérképezése és elemzése. A feladat magában foglalja a különböző metrikagyűjtési és vizualizációs technológiák (pl. Prometheus, OpenTelemetry) összehasonlító vizsgálatát, valamint a moq-rs architektúrájához leginkább illeszkedő megoldás kiválasztását. A hallgatónak meg kell vizsgálnia, hogyan gyűjthetők hatékonyan rendszerszintű és alkalmazásszintű adatok elosztott környezetben. Az eredményeket mutassa be egy implementációval, vizualizálja, és értékelje a látható eredményeket.

Tanév: 2025/2026. tanév, I. félév

1. Bevezetés

A modern videóstreaming felhasználói egyre magasabb elvárásokat támasztanak a tartalom minőségével és valós idejű elérhetőségével szemben. Az olyan interaktív alkalmazásoknál, mint az élő közvetítések, a hagyományos protokollok késleltetése már zavaró lehet – például amikor a szomszéd előbb ünnepli a gólt, mint ahogy mi látjuk. Erre kínál megoldást a Media over QUIC (MoQ) [1], amely a QUIC protokoll [2] gyorsaságát és rugalmasságát használja ki a médiaátvitelre, felváltva az elavult TCP-alapú megoldásokat.

A Media over QUIC nem csupán egy új szállítási protokoll, hanem egy teljesen új hálózati architektúrát is feltételez, amely a publikáló-feliratkozó modellre épül. Ebben az architektúrában három fő szereplőt különböztetünk meg: a tartalom előállítóját (publisher), a fogyasztót (subscriber) és a közvetítő hálózatot alkotó relay csomópontokat. A relayek feladata a tartalom továbbítása és gyorsítótárazása, hasonlóan a hagyományos CDN (Content Delivery Network) topológiákhoz.

Egy ilyen elosztott rendszerben a hálózat állapotának és teljesítményének nyomon követése nem egyszerű. Ez köszönhető részben a QUIC titkosításának TLS-sel, ami miatt nem tudunk belehallgatni az adatfolyamokba, részben pedig annak, hogy a hálózat résztvevői mind külön-külön QUIC-kapcsolatot tartanak fent, ezzel is bonyolítva a megfigyelést. A rendszer komplexitása miatt nehéz átfogó képet kapni a működésről, ezért érdemes lehet megvizsgálni a megfigyelhetőség (observability) kialakítását. Jelenleg a MoQ implementációk megfigyelhetősége korlátozott, ami nehezíti, hogy jól felhasználható statisztikákat készítsünk. A feladat célja a moq-rs [3] implementáció mérési lehetőségeinek vizsgálata. A munka során:

- Feltérképezésre kerülnek a lehetséges mérési pontok a moq-rs architektúrájában.
- Összehasonlításra kerülnek a különböző telemetriagyűjtési technológiák.
- Tervezésre és implementálásra kerül egy moduláris mérési rendszer.
- Az eredmények vizualizációja is megtörténik, hogy azok az üzemeltetők számára is értelmezhetők legyenek.
- A mérési eredmények helyességének ellenőrzése és a rendszer működésének validálása is megtörténik.

2. Háttér

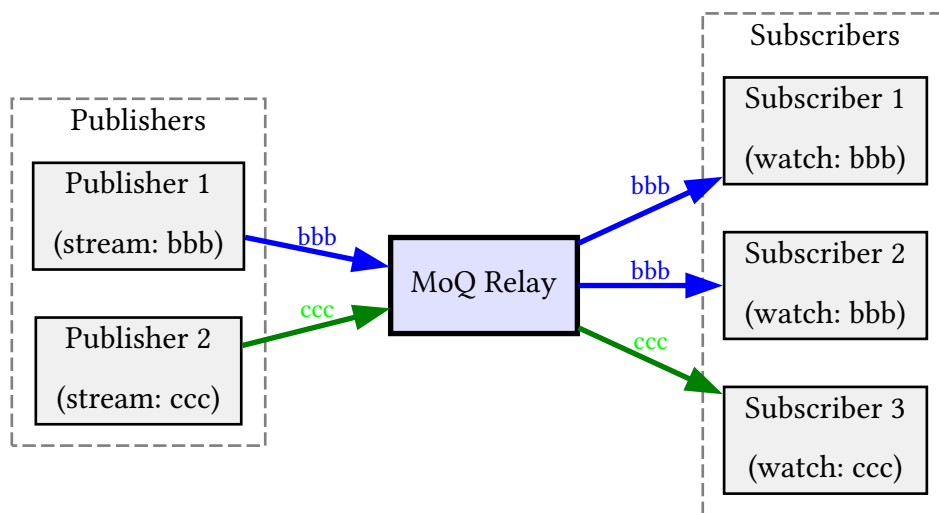
2.1. A MoQ-ökoszisztéma

A Media over QUIC (MoQ) architektúra (1. ábra) alapvetően három szereplőt különböztet meg: a **publisher**-t, aki a tartalom előállítója (stream forrása, kamera stb.), a **relay**-t, ami a közvetítő szerver, amely gyorsítótárazza és továbbítja az adatokat, alkotva a CDN (Content Delivery Network) gerincét, valamint a **subscriber**-t, ami a fogyasztó (pl. videolejátszó).

A szereplőkön kívül fontos megemlíteni a MoQ adatmodelljének elemeit is. A **namespace** (névtér) a streamek logikai csoportosítását szolgálja, segítve a tartalmak rendszerezésében (pl. bbb vagy ccc). A **stream** egy önálló tartalomfolyam (pl. egy videóadás), amelyet mindig egy névtér azonosít. A streamen belüli logikai egység a **track** (sáv), amely lehet például külön kép- és hangsáv, vagy különböző felbontás. A legkisebb átviteli egységek a **group** és az **object**. A sávok csoportokra (group), azok pedig objektumokra (object) vannak bontva. Egy objektum

lehet például egy részlete a képnek (pár képkocka) vagy a hangsávnak, és a hálózati átvitel ezen a szinten történik.

A feladat alapjául szolgáló moq-rs projekt ezt az architektúrát valósítja meg Rust nyelven. A projekt több, egymásra épülő komponensből áll: a rendszer magját a **moq-transport** adja, amely a MoQ protokoll alapvető alkotóelemeit valósítja meg QUIC felett. A hálózat gerincét a **moq-relay**, a központi médiatovábbító szerver biztosítja, míg a **moq-pub** és **moq-sub** a referencia kliensimplementációk (publisher és subscriber).



1. ábra: A Media over QUIC (MoQ) architektúra logikai felépítése több stream esetén.

2.2. Megfigyelhetőség és mérés

A megfigyelhetőségnek három elterjedt módját ismerjük. A **logging** (naplózás) konkrét események rögzítését jelenti, ami leírja egy program működését, és elsődleges célja a hibakeresés és az auditálás. A **tracing** (nyomkövetés) jellemzően egy kérés, hívás, vagy egy csomag útjának nyomon követése a rendszer különböző komponensein keresztül. A **metrics** (metrikák) pedig számszerű adatok gyűjtése idősoros formában globálisan.

Ezek közül a jelenlegi feladat szempontjából a metrikák a legfontosabbak, mivel ezek adnak átfogó képet a rendszer globális állapotáról és teljesítményéről anélkül, hogy elvesznénk az egyedi csomagok részleteiben. A metrikáknak három fő típusát különböztetjük meg: a **counter**-t, amely csak növelhető (pl. összes fogadott bájtt) és újraindításkor nullázódik, a **gauge**-t, amely egy fel-le változó érték és egy pillanatnyi állapotot tükröz (pl. aktív kapcsolatok száma), valamint a **histogram**-ot, amely eloszlások mérésére szolgál (pl. válaszidők).

A metrikák gyűjtésére két alapvető architektúrális minta létezik. A **pull modell** (pl. Prometheus [4]) esetében a monitorozó szerver periodikusan lekérdezi („scrape”) az alkalmazás végpontját. Ennek előnye a központi vezérlés, hátránya a hálózati elérés nehézségei lehetnek. A **push modell** (pl. Graphite, OpenTelemetry OTLP [5]) esetében az alkalmazás küldi el az adatokat, ami egyszerűbb hálózati szempontból, de nehezebben skálázható a terhelés.

Bár a hálózat dinamikus felépítése indokolhatná a push modellt, a Prometheus elterjedtsége és egyszerűsége miatt a fejlesztés során mégis a pull modell került kiválasztásra.

2.3. Használt technológiák

Mivel a `moq-rs` teljes egészében Rust nyelven íródott, a választott technológiának rendelkeznie kell megfelelő Rust támogatással.

A **Prometheus** jelenleg az ipari standard a metrikák tárolására és lekérdezésére. Nagyon elterjedt, nagy ökoszisztémával rendelkezik, amit egy erős lekérdezési nyelv, a PromQL támogat.

Az **OpenTelemetry (OTel)** egy újabb szabvány, amely egyesíti a trace-ek, logok és metrikák gyűjtését, azonban használata bonyolult és körülményes. Előnye, hogy a backend (ami akár Prometheus is lehet) könnyen cserélhető.

Ezek miatt a Prometheus lett használva közvetlenül, mert egyrészt egyszerűbb a használata, másrészt kevesebb erőforrást is igényel, valamint most kifejezetten a metrikákon van a hangsúly.

A két technológia összehasonlítását az 1. táblázat tartalmazza.

Tulajdonság	Prometheus	OpenTelemetry
Modell	Pull (alapértelmezett)	Push (OTLP)
Adattípusok	Metrika	Trace, Log, Metrika
Integráció komplexitása	Alacsony	Magas
Rust támogatás	Stabil	Fejlesztés alatt

1. táblázat: A Prometheus és az OpenTelemetry összehasonlítása.

A fejlesztéshez kellett választani egy Prometheus könyvtárat. A Rust ökoszisztémában a mérésre több megoldás létezik:

- **prometheus-client:** A Prometheus projekt hivatalos Rust klienskönyvtára [6]. Ez nem egy generikus facade, hanem egy típusbiztos, közvetlen implementáció, amely saját regisztert (Registry) kezel. Előnye a szoros integráció a Prometheus adatmodellel és a nagy teljesítmény.
- **metrics:** Egy népszerű, generikus könyvtár [7]. Bár rugalmas, a `prometheus-client` közvetlenebb kontrollt biztosít a metrikák felett.
- **opentelemetry:** Az OTel teljes implementációja [5]. Bár nagy tudású, nehezebb a használata és nagyobb overhaddel jár.

A `moq-rs` fejlesztése során a `prometheus-client` könyvtárat használtam, mivel ennek használata a leghatékonyabb és legegyszerűbb Prometheus formátumú adatok kezelésére.

A vizualizációhoz a Grafanat [8] választottam, amely egy nyílt forráskódú vizualizációs platform, ami lehetővé teszi a metrikák lekérdezését, megjelenítését és vizualizációját. Képes számos adatforráshoz, köztük a Prometheushoz is kapcsolódni, így ideális választás a gyűjtött adatok megjelenítésére. A segítségével létrehozhatunk dinamikus dashboardokat, amelyek

valós időben mutatják a rendszer állapotát, például a sávszélesség-használatot vagy a kapcsolatok számát.

3. Metrikagyűjtés beépítése a moq-rs-be

A rendszer tervezésekor az elsődleges szempont, hogy az architektúra minél függetlenebb legyen az eredeti moq-rs kódtól, elősegítve ezzel a későbbi integrációt a hivatalos kódbázisba egy egyszerűsített pull request (PR) formájában. Ezen kívül nagy hangsúlyt fektettem a modularitásra is, hogy könnyen használható legyen a változtatás. A méréseket végző kód ezért egy különálló könyvtárba, a moq-metrics-be került kiszervezésre.

Ez a komponens felelős a metrikák regisztrálásáért és az exporterek kezeléséért. Gyakorlatilag a Prometheus backendet köti össze az alkalmazás többi részével. Főbb feladatai:

- a prometheus-client használata,
- a Prometheus exporter elindítása egy konfigurálható HTTP porton,
- hálózati metrikák gyűjtése,
- rendszermetrikák (CPU, memória) periodikus gyűjtése.

A tervezés fontos lépése volt a releváns metrikák meghatározása. Nem az volt a cél, hogy „minden legyen mérve”, hanem hogy olyan mutatókat lehessen mérni, amelyekből következtetni lehet a hálózat minőségére és a rendszer korlátaira. A lib.rs-ben definiált metrikák négy fő kategóriába sorolhatók:

1. **Forgalmi adatok:** A átvitt adatmennyiség (bájtok) és objektumok száma, mind a publikálók, mind a feliratkozók irányába. Ez ad képet a rendszer tényleges terheléséről.
2. **Kapcsolatok állapota:** Az aktív QUIC kapcsolatok, publikálók és feliratkozók száma, valamint a hálózati minőség mutatói (pl. RTT, csomagvesztés).
3. **MoQ specifikus metrikák:** A protokoll szintű entitások, mint a meghirdetett és feliratkozott sávok követése.
4. **Rendszererőforrások:** A CPU és memória felhasználása mind folyamat, mind rendszer-szinten, ami kritikus a skálázódás és a hardveres korlátok felismerése szempontjából.

A projektben a **pull modell** került kiválasztásra a Prometheus széleskörű támogatottsága miatt. Minden moq-rs komponens (relay, publisher, subscriber) elindít egy HTTP szerveret, ahonnan a Prometheus szerver begyűjtheti az aktuális metrikákat.

4. Megvalósítás

A megvalósítás során a prometheus-client crate API-ja került felhasználásra a megfelelő mérési pontokon a moq-rs kódbázisában.

A lib.rs számos metrikát definiál, amelyek közül a teljesség igénye nélkül néhány fontosabb példa:

- **moq_relay_bytes_received_from_publisher (Counter):** A publikálóktól fogadott adat-mennyiség bájtokban.
- **moq_relay_quic_connections_active (Gauge):** Az éppen élő QUIC kapcsolatok száma.
- **moq_relay_active_subscribed_tracks (Gauge):** Az aktív feliratkozások száma névtér (namespace) szerint bontva.
- **moq_relay_quic_rtt_milliseconds (Gauge):** A QUIC kapcsolatok körkörös késleltetése (RTT) milliszekundumban.
- **process_cpu_usage_percent (Gauge):** A folyamat CPU használata százalékban.

A 2. táblázat tartalmazza az összes eddig implementált metrikát.

Metrika	Típus	Mérési pont
moq_relay_announced_tracks	Counter	Relay
moq_relay_announced_tracks_current	Gauge	Relay
moq_relay_active_subscribed_tracks	Gauge	Relay
moq_relay_objects_sent	Counter	Relay
moq_relay_bytes_received_from_publisher	Counter	Relay
moq_relay_bytes_sent_to_subscriber	Counter	Relay
moq_relay_active_publishers	Gauge	Relay
moq_relay_quic_rtt_milliseconds	Gauge	Relay
moq_relay_quic_connections_active	Gauge	Relay
moq_relay_quic_lost_packets	Gauge	Relay
moq_relay_quic_sent_packets	Gauge	Relay
process_cpu_usage_percent	Gauge	Minden
process_memory_bytes	Gauge	Minden
system_cpu_usage_percent	Gauge	Minden
system_memory_bytes	Gauge	Minden
system_memory_available_bytes	Gauge	Minden
subscriber_objects_received	Counter	Subscriber
subscriber_bytes_received	Counter	Subscriber
subscriber_active_tracks	Gauge	Subscriber
publisher_objects_sent	Counter	Publisher
publisher_bytes_sent	Counter	Publisher
publisher_active_tracks	Gauge	Publisher

2. táblázat: A lib.rs-ben implementált metrikák összefoglalása.

A moq-metrics könyvtárat egyszerűen hozzá lehet adni a többi moq-rs komponenshez, és a korábban említett singleton mintának köszönhetően csak egy-egy sort függvényhívást kell betűzni a metrikák növeléséhez. Fontos persze, hogy a HTTP szervereket a komponens felfutásakor elindítsuk, de erre is van globálisan elérhető, publikus függvény.

4.1. Implementációs részletek

A fejlesztés során több érdekes implementációs kérdés is felmerült, amelyek közül a legfontosabbak:

4.1.1. Címkézés

A metrikák nem csupán egyszerű számlálók, címkékkel is el lehet őket látni. Például a sávszélesség mérésekor a namespace címke jelöli, hogy melyik streamhez tartozik az adat. Ez teszi lehetővé a Grafanában a későbbi szűrést és a részletes bontást (pl. melyik stream „eszi” a legtöbb sávszélességet, vagy melyik relay van földrajzilag leterhelve), ami sokat segít a mutatók összeállításában. Persze a jelenleg implementált címkéken kívül igény szerint lehetne még implementálni, pl. track ID, IP, port stb., ami hasznos lehet.

4.1.2. Frissítési stratégiák

A fejlesztés egyik központi kérdése az volt, hogy mikor és hogyan frissüljenek a metrikák értékei. Két eltérő megközelítés is alkalmazásra került a metrikák természetétől függően:

A **QUIC metrikák** esetében a kapcsolatok állapota (pl. aktív kapcsolatok száma, RTT) a moq-transport belső memóriájában folyamatosan rendelkezésre áll. Felesleges lenne egy külön szálon, másodpercenként lekérdezni és másolni ezeket az adatokat, ha a Prometheus csak 15 másodpercenként kéri le őket. Ezért itt a „lazy” kiértékelés történik: a metrikák frissítése csak akkor történik meg, amikor beérkezik a HTTP GET kérés a /metrics végpontra. Ezt a HTTP szerver handlerében lett megvalósítva, amely közvetlenül a válaszadás előtt olvassa ki az aktuális állapotot, minimalizálva a CPU terhelést és a kontextusváltásokat.

Ezzel szemben a **rendszermetrikák**, különösen a CPU terhelés, gyorsan változnak. Ha itt is igény szerint történne a lekérés, és csak a begyűjtés pillanatában (pl. 5 másodpercenként) lenne mérve a terhelés, könnyen lemaradhatna egy-egy kicsúcsosodás. Ezért a sysinfo [9] alapú méréshez egy dedikált, háttérben futó tokio::task indul (tokio::spawn [10]). Ez a ciklus sűrűbben mintavételez, és egy belső állapotban tárolja az aggregált értékeket. Amikor a Prometheus lekérdez, már ezt a pontosabb, simított értéket kapja meg. Bár ez némi plusz erőforrást igényel, elengedhetetlen a valós teljesítményképhez.

Ez a hibrid megoldás biztosítja az egyensúlyt, a lassabban változó hálózati metrikák és a gyorsabban változó rendszermetrikák így optimálisan kerülnek kiértékelésre.

4.1.3. Tervezési minták

A fejlesztés során két fontos tervezési minta is alkalmazásra került. Az egyik a **Singleton** minta: bár a globális változók használata általában kerülendő, ebben az esetben a GLOBAL_METRICS singleton (a once_cell::sync::Lazy [11] segítségével) jelentősen egyszerűsítette a hívásokat. Így nem kellett minden függvénynek paraméterként átadni a metrika registryt, hanem bárholnán elérhetővé váltak a számlálók (pl. moq_metrics::increment_active_connections()).

A másik fontos minta a **RAII** (Resource Acquisition Is Initialization). A kapcsolatok és sávok számának mérésekor fontos, hogy a számlálók értéke csökkenjen, amikor az erőforrás felszabadul. A manuális `dec()` hívások helyett RAII guardokat implementáltam. Ezek olyan struktúrák, amelyek `Drop` implementációja automatikusan csökkenti a metrikát, amikor a változó kikerül a scopeból.

4.1.4. Makrók használata

Példa a `moq-metrics/src/lib.rs`-ből (1. kódrészlet), ahol egy Rust makró segítségével kerültek definiálásra a metrikák. Ez a megoldás jelentősen csökkenti a kódismétlést és egyszerűsíti új metrikák felvételét:

```
// A makró használata: itt soroljuk fel az összes metrikát
define_metrics! {
    objects_sent:
        Counter<u64>,
        "moq_relay_objects_sent",
        "Total objects sent",
    // ... további metrikák ...
}
```

1. kódrészlet: A metrikákat definiáló makró használata.

A fenti makróhívás a fordítási időben a következő (egyszerűsített) kódra bomlik ki (2. kódrészlet), ami jól szemlélteti, mennyi „boilerplate” kódot spóroltunk meg:

```
// A generált kód (kibontva)
#[derive(Clone)]
pub struct MoqMetrics {
    pub objects_sent: Counter<u64>,
    // ... többi mező
}

impl MoqMetrics {
    pub fn new(registry: &mut Registry) -> Self {
        let objects_sent = <Counter<u64>>::default();
        registry.register(
            "moq_relay_objects_sent",
            "Total objects sent",
            objects_sent.clone()
        );
        // ... többi regisztráció

        Self {
            objects_sent,
            // ... többi mező
        }
    }
}
```

2. kódrészlet: A makró által generált kód egyszerűsített változata.

A metrikák növelése pedig a kód megfelelő pontjain történik (3. kódrészlet), például a `moq-transport/src/session/subscriber.rs`-ben, amikor adat érkezik:

```
// Példa a számláló növelésére (subscriber.rs)
pub fn add_bytes_received(count: u64) {
    GLOBAL_METRICS
        .metrics
        .bytes_received_from_publisher
        .inc_by(count);
}

// ...
// A fogadott adat méretének hozzáadása a metrikához
moq_metrics::add_bytes_received(data.len() as u64);
```

3. kódrészlet: Példa a metrikák növelésére a kódban.

4.2. Fejlesztési nehézségek és tapasztalatok

A fejlesztés során több kihívás is felmerült:

A **mérési pontok helyes kiválasztása** során a leggyakoribb hiba a „dupla számolás” volt. Mivel bizonyos függvények a kód több pontjáról vagy többször is meghívódhatnak egy esemény bekövetkezetekor, nehéz volt megtalálni azt az egyetlen pontot, ahol a metrikát biztonságosan lehet növelni. Emiatt például az aktív feliratkozott sávokra vonatkozó metrika eleve a feliratkozókra vonatkozott, de kiderült, hogy mindig sávonként lehet csak növelni, és egy streamhez alapértelmezetten nem is egy, hanem több is tartozik, mert mindig van egy „init” track a rendes médiafolyam mellett.

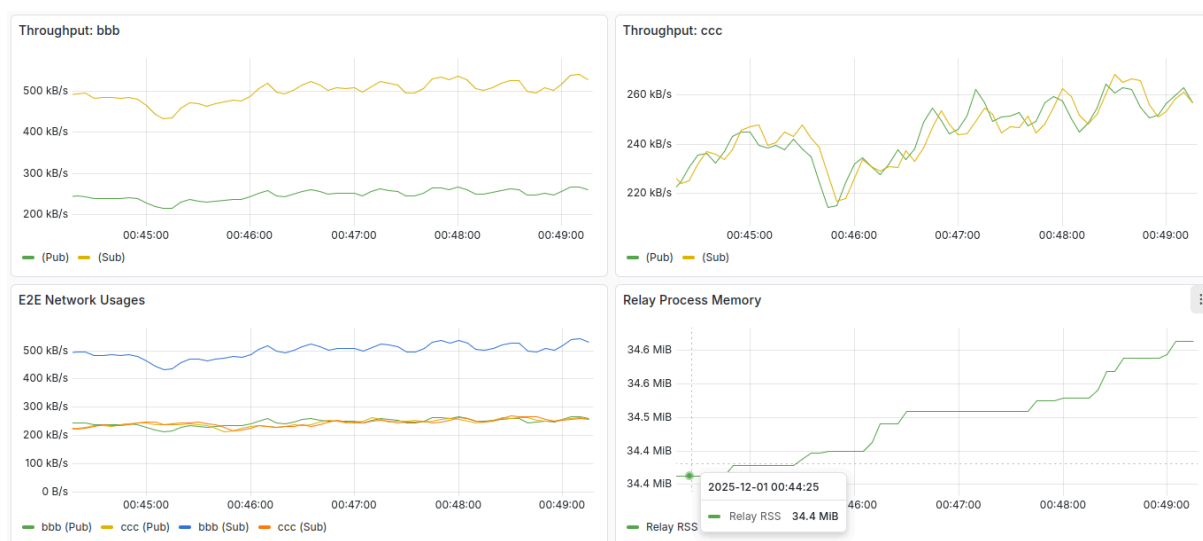
A korábban említett **„on-demand” QUIC-mérésnek** kezdetben volt egy technikai akadály. A quinn könyvtárban [12] a kapcsolatok statisztikáinak lekérdezése (`conn.stats()`) belső zárolással (lock) jár. Ez azt jelentené, hogy nagy számú QUIC kapcsolat esetén a Prometheus lekérdezés (scrape) ideje alatt a rendszernek egyszerre kellene iterálnia az összes kapcsolaton, ami blokkolhatta volna az új kapcsolatok fogadását. Ezt végül sikerült kiküszöbölni: a metrikák lekérdezésekor a kapcsolatok listája lemásolásra kerül (Arc segítségével) a zárolás alatt, így a QUIC-kapcsolatok lekérése már nem blokkol. Ennek ellenére ez a dilemma figyelmet igényel, ugyanis nem minden esetben egyértelmű, hogy pontosan mit történik több ezer QUIC kapcsolat esetén.

A metrikák bevezetése segített beazonosítani **rejtett hibákat** is. A `moq_relay_bytes_received_from_publisher` számláló növekedéséből látszott, hogy a relay akkor is fogadott adatot a publishertől, amikor már nem volt aktív feliratkozó. Ez egy olyan hibát jelzett, ami valószínűleg olyan helyzetben történik, ahol a DROP esemény nem került megfelelően átadásra a publisher felé. Ez az eset is mutatja, hogy a metrikagyűjtés valóban hasznosnak bizonyul.

5. Vizualizáció és elemzés

A begyűjtött metrikák megjelenítésére a Grafana került kiválasztásra, mivel natívan támogatja a Prometheus és rugalmasan testreszabható.

A dashboard úgy lett kialakítva, hogy a legfontosabb mutatókat és a belőlük származtatott értékeket jelenítse meg, átfogó képet adva a rendszer működéséről. A vizualizáció segítségével valós időben követhető nyomon a publikálók és feliratkozók általi kommunikáció, valamint a hálózati sávszélességigény. Látható a forgalom megoszlása a különböző streamek között, ami segít azonosítani a legnépszerűbb tartalmakat vagy a szűk keresztmetszeteket. Emellett a dashboardon megjelenik a relay folyamat és a rendszer erőforrás-használata (CPU, memória) is, ami hasznos lehet a teljesítménybeli problémák diagnosztizálásához. A közzétett és feliratkozott sávok (tracks) számának összevetése pedig segít a „szivárgó” feliratkozások vagy inkonzisztens állapotok felfedezésében. Az implementált metrikák segítségével számos további panel megvalósítható igényeknek megfelelően.

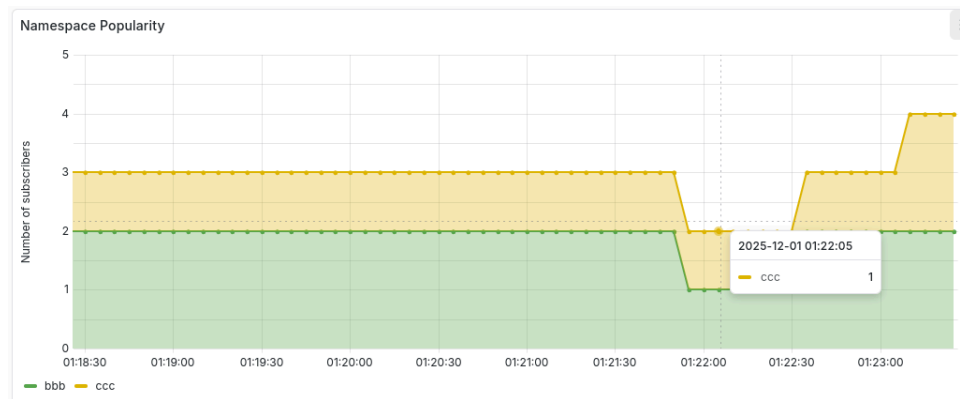


2. ábra: A megvalósított Grafana dashboard egy részlete.

5.1. Eredmények értelmezése

A mérések során a rendszer erőforrásigényét és a hálózati stabilitást vizsgáltuk. Látszódik, ha változik a feliratkozók vagy a publisherek száma, új streammel bővül a médiafolyam, és az is látszódik, hogy hogyan változik a terhelés ezek függvényében. Jól látszódik az is, ha valamelyik stream a szokásosnál több erőforrást használ, megnő a sávszélesség, hirtelen változás esetén a késleltetés is növekedik.

A 2. ábra pár fontosabb panelt tartalmaz. A felső kettő azt mutatja, hogy mekkora az egyes streamek (bbb és ccc) által használt sávszélesség a hálózatban. A bal alsó panelen ezek össze-sítése látható. Az képernyőkép készítése előtt szándékosan változtattam a feliratkozók számát, ez sávszélességbeli ingadozást okoz, ami a grafikonokon is megjelenik. A jobb alsó panelen egy egyszerűbb metrika látható, a relay által használt memória méretét. Ezen az látszódik, hogy az események számának növekedésével (új feliratkozó, új publisher) a memórafogyasztás is növekedik, bár kis mértékben.



3. ábra: A különböző streamek használatának megoszlása.

A 3. ábra mutat egy komplexebb panelt, amin látható, hogy valós időben hogyan változott a feliratkozók száma névterekre lebontva.

6. Fejlesztési lehetőségek és összegzés

A féléves munka során sikeresen implementálásra került egy alapvető mérési modul a moq-rs-ben, de számos területen van még lehetőség a fejlődésre.

A jelenlegi metrikapalettát még lehet bővíteni igény szerint, például a puffer és cache megfigyelésével, egyéb, lokáció alapú metrikákkal, stb.

A fejlesztett moq-metrics könyvtár jelenleg a saját forkban él. A hosszú távú cél, hogy a tapasztalatok alapján egy letisztult, egyszerűsített demó változat készüljön, amely Pull Request formájában visszakerülhet az eredeti moq-rs projektbe. Ezáltal a közösség számára is elérhetővé válna egy alapvető, de bővíthető megfigyelhetőségi réteg.

Vannak olyan mérendő adatok, amik nem nyerhetők ki ezzel az egyszerű, tiszta Prometheus implementációval. Érdemes lehet megvizsgálni a lehetőségét annak, hogy az egyes objektumokhoz időbélyeget fűzzünk hozzá, amivel mérhetővé válna az E2E késleltetés mérése. Erre létezik is már formátum a MoQ draftjában, amit lehetne használni, de még nincs implementálva.

A laboratóriumi munka során áttekintésre kerültek a Media over QUIC technológia kihívásai és a modern megfigyelhetőségi eszközök. Tervezésre és megvalósításra került egy moduláris mérési könyvtár, amely képes valós időben információt szolgáltatni a rendszer működéséről. A mérések eredményei igazolták, hogy lehetséges és hasznos a metrikák gyűjtése MoQ-környezetben is. A hasznosságot a korábban említett megtalált implementációs hiba is alátámasztja.

Irodalomjegyzék

- [1] L. Curley, K. Pugin, és S. Nandakumar, „Media over QUIC Transport”, Internet-Draft, 2024. Elérés: 2025. december 3. [Online]. Elérhető: <https://datatracker.ietf.org/doc/draft-ietf-moq-transport/>
- [2] J. Iyengar és M. Thomson, „QUIC: A UDP-Based Multiplexed and Secure Transport”, RFC RFC9000, máj. 2021. Elérés: 2025. december 3. [Online]. Elérhető: <https://www.rfc-editor.org/rfc/rfc9000.html>
- [3] Cloudflare, „moq-rs Repository”. Elérés: 2025. december 3. [Online]. Elérhető: <https://github.com/cloudflare/moq-rs>
- [4] The Prometheus Authors, „Prometheus Documentation”. 2024. Elérés: 2025. december 3. [Online]. Elérhető: <https://prometheus.io/docs/introduction/overview/>
- [5] OpenTelemetry Authors, „OpenTelemetry Documentation”. 2024. Elérés: 2025. december 3. [Online]. Elérhető: <https://opentelemetry.io/docs/>
- [6] Prometheus Team, „prometheus-client”. Elérés: 2025. december 3. [Online]. Elérhető: https://docs.rs/prometheus-client/latest/prometheus_client/
- [7] T. Lawrence, „metrics”. Elérés: 2025. december 3. [Online]. Elérhető: <https://docs.rs/metrics/latest/metrics/>
- [8] Grafana Labs, „Grafana Documentation”. 2024. Elérés: 2025. december 3. [Online]. Elérhető: <https://grafana.com/docs/>
- [9] G. Gomez, „sysinfo”. Elérés: 2025. december 3. [Online]. Elérhető: <https://docs.rs/sysinfo/latest/sysinfo/>
- [10] Tokio Contributors, „Tokio Runtime”. Elérés: 2025. december 3. [Online]. Elérhető: <https://tokio.rs/>
- [11] A. Kladov, „once_cell”. Elérés: 2025. december 3. [Online]. Elérhető: https://docs.rs/once_cell/latest/once_cell/
- [12] Quinn Contributors, „quinn”. Elérés: 2025. december 3. [Online]. Elérhető: <https://github.com/quinn-rs/quinn>